Knowledge Graph: 서브그래프와 컨텍스트 변환

개요

Knowledge Graph(KG)는 엔티티(노드)와 관계(엣지)로 구성된 그래프 구조입니다. RAG에서 KG를 활용하면 단순 텍스트 검색보다 더 풍부한 맥락을 제공할 수 있습니다.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    Knowledge Graph 기반 RAG 흐름                         │
│                                                                          │
│  ┌─────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐  │
│  │  Query  │ -> │ Entity       │ -> │ Subgraph     │ -> │ Context      │  │
│  │         │    │ Extraction   │    │ Retrieval    │    │ Generation   │  │
│  │ "김대리 │    │              │    │              │    │              │  │
│  │  프로젝 │    │ [김대리]     │    │ 관련 노드/   │    │ 자연어       │  │
│  │  트는?" │    │ [프로젝트]   │    │ 엣지 추출    │    │ 문맥 변환    │  │
│  └─────────┘    └──────────────┘    └──────────────┘    └──────────────┘  │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Part 1: Knowledge Graph 구조

1.1 기본 구성 요소

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Knowledge Graph 구조                            │
│                                                                          │
│    ┌─────────┐                                      ┌─────────┐          │
│    │  Node   │                                      │  Node   │          │
│    │ (Entity)│                 ┌──────────────┐     │ (Entity)│          │
│    │         │────────>│    Edge      │──────────>│         │          │
│    │  - id   │         │ (Relation)   │           │  - id   │          │
│    │  - type │         │              │           │  - type │          │
│    │  - props│         │  - type      │           │  - props│          │
│    └─────────┘         │  - props     │           └─────────┘          │
│                        │  - weight    │                                │
│                        └──────────────┘                                │
│                                                                          │
│   예시:                                                                  │
│   [Person:김철수] ────WORKS_AT────────> [Company:ABC Corp]              │
│        |                                        |                       │
│        |                                        |                       │
│     MANAGES                                  LOCATED_IN                 │
│        |                                        |                       │
│        ▼                                        ▼                       │
│   [Project:알파]                            [Location:서울]              │
│        |                                                                │
│        |                                                                │
```

```
│     USES_TECH                                                    │
│        │                                                         │
│        ▼                                                         │
│   [Technology:Python]                                            │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

1.2 노드/엣지 데이터 구조

```python
# knowledge_graph/models.py
from dataclasses import dataclass, field
from typing import Dict, List, Any, Optional
from enum import Enum
import json

class NodeType(Enum):
    PERSON = "Person"
    ORGANIZATION = "Organization"
    PROJECT = "Project"
    TECHNOLOGY = "Technology"
    DOCUMENT = "Document"
    CONCEPT = "Concept"
    EVENT = "Event"
    LOCATION = "Location"
    PRODUCT = "Product"

class RelationType(Enum):
    # 인물 관계
    WORKS_AT = "WORKS_AT"
    MANAGES = "MANAGES"
    REPORTS_TO = "REPORTS_TO"
    COLLABORATES_WITH = "COLLABORATES_WITH"

    # 프로젝트 관계
    WORKS_ON = "WORKS_ON"
    OWNS = "OWNS"
    CONTRIBUTES_TO = "CONTRIBUTES_TO"

    # 기술 관계
    USES_TECH = "USES_TECH"
    REQUIRES = "REQUIRES"
    IMPLEMENTS = "IMPLEMENTS"

    # 문서 관계
    AUTHORED = "AUTHORED"
    REFERENCES = "REFERENCES"
    DESCRIBES = "DESCRIBES"

    # 일반 관계
    RELATED_TO = "RELATED_TO"
    PART_OF = "PART_OF"
```

```python
    LOCATED_IN = "LOCATED_IN"
    OCCURRED_AT = "OCCURRED_AT"

@dataclass
class Node:
    """그래프 노드 (엔티티)"""
    id: str
    type: NodeType
    name: str
    properties: Dict[str, Any] = field(default_factory=dict)
    embedding: Optional[List[float]] = None

    def to_dict(self) -> dict:
        return {
            'id': self.id,
            'type': self.type.value,
            'name': self.name,
            'properties': self.properties,
        }

    def to_text(self) -> str:
        """노드를 텍스트로 변환"""
        props_str = ', '.join(f"{k}: {v}" for k, v in self.properties.items())
        return f"{self.type.value} '{self.name}'" + (f" ({props_str})" if
props_str else "")

@dataclass
class Edge:
    """그래프 엣지 (관계)"""
    source_id: str
    target_id: str
    type: RelationType
    properties: Dict[str, Any] = field(default_factory=dict)
    weight: float = 1.0

    def to_dict(self) -> dict:
        return {
            'source': self.source_id,
            'target': self.target_id,
            'type': self.type.value,
            'properties': self.properties,
            'weight': self.weight,
        }

    def to_text(self, source_name: str, target_name: str) -> str:
        """엣지를 텍스트로 변환"""
        # 관계 타입을 자연어로
        relation_text = self.type.value.replace('_', ' ').lower()
        return f"{source_name} {relation_text} {target_name}"

@dataclass
class Triplet:
    """트리플렛 (Subject, Predicate, Object)"""
    subject: Node
```

```
    predicate: Edge
    object: Node

    def to_text(self) -> str:
        """트리플렛을 자연어 문장으로"""
        return self.predicate.to_text(self.subject.name, self.object.name)
```

1.3 전체 그래프 예시

```
# 실제 Knowledge Graph 예시
"""
┌──────────────────────────────────────────────────────────────────────────┐
│                        회사 프로젝트 Knowledge Graph                      │
│                                                                          │
│                              [ABC Corp]                                  │
│                             /    |    \                                  │
│                   WORKS_AT/     |     \LOCATED_IN                        │
│                          /      |      \                                 │
│                    [김철수]  [이영희]   [서울 본사]                       │
│                    /    \      |                                         │
│              MANAGES/      \    |WORKS_ON                                │
│                   /    WORKS_ON \|                                       │
│                  /              \|                                       │
│           [알파 프로젝트]────────[베타 프로젝트]                           │
│                |      \              |                                  │
│            USES_TECH   \REFERENCES   |USES_TECH                          │
│                |        \            |                                  │
│           [Python]    [설계문서]  [Kubernetes]                           │
│                |          |          |                                  │
│           REQUIRES    DESCRIBES   REQUIRES                               │
│                |          |          |                                  │
│           [FastAPI]   [알파 프로젝트] [Docker]                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
"""

# 위 그래프를 코드로 표현
nodes = [
    Node("company_1", NodeType.ORGANIZATION, "ABC Corp",
         {"industry": "IT", "founded": 2010, "employees": 500}),

    Node("person_1", NodeType.PERSON, "김철수",
         {"role": "Tech Lead", "department": "개발팀", "experience": 8}),
    Node("person_2", NodeType.PERSON, "이영희",
         {"role": "PM", "department": "기획팀", "experience": 5}),

    Node("project_1", NodeType.PROJECT, "알파 프로젝트",
         {"status": "진행중", "start_date": "2024-01", "priority": "high"}),
    Node("project_2", NodeType.PROJECT, "베타 프로젝트",
         {"status": "기획", "start_date": "2024-06", "priority": "medium"}),
```

```
    Node("tech_1", NodeType.TECHNOLOGY, "Python", {"version": "3.11"}),
    Node("tech_2", NodeType.TECHNOLOGY, "FastAPI", {"version": "0.109"}),
    Node("tech_3", NodeType.TECHNOLOGY, "Kubernetes", {"version": "1.28"}),
    Node("tech_4", NodeType.TECHNOLOGY, "Docker", {"version": "24.0"}),

    Node("doc_1", NodeType.DOCUMENT, "설계문서",
         {"type": "technical", "pages": 45}),

    Node("loc_1", NodeType.LOCATION, "서울 본사",
         {"address": "강남구", "floor": 15}),
]

edges = [
    Edge("person_1", "company_1", RelationType.WORKS_AT),
    Edge("person_2", "company_1", RelationType.WORKS_AT),
    Edge("company_1", "loc_1", RelationType.LOCATED_IN),

    Edge("person_1", "project_1", RelationType.MANAGES),
    Edge("person_1", "project_2", RelationType.WORKS_ON),
    Edge("person_2", "project_2", RelationType.WORKS_ON),

    Edge("project_1", "tech_1", RelationType.USES_TECH),
    Edge("project_2", "tech_3", RelationType.USES_TECH),
    Edge("tech_1", "tech_2", RelationType.REQUIRES),
    Edge("tech_3", "tech_4", RelationType.REQUIRES),

    Edge("doc_1", "project_1", RelationType.DESCRIBES),
    Edge("project_2", "doc_1", RelationType.REFERENCES),
]
```

Part 2: 서브그래프 추출

2.1 서브그래프란?

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          서브그래프 추출 개념                             │
│                                                                          │
│   전체 그래프 (Full Graph)              서브그래프 (Subgraph)            │
│                                                                          │
│      [A]──[B]──[C]                          [B]                          │
│       │    │    │                          / | \                        │
│       │    │    │          Query:         /  |  \                       │
│      [D]──[E]──[F]        "B 관련"    [A]─┘  |   └─[C]                  │
│       │    │    │            =>             |                            │
│       │    │    │                          [E]                          │
│      [G]──[H]──[I]                                                       │
│                                                                          │
│   - 전체 그래프는 너무 큼                - 쿼리와 관련된 부분만 추출     │
│   - LLM 컨텍스트에 다 넣을 수 없음       - 1-hop, 2-hop 이웃 노드 포함   │
│                                          - 컨텍스트 윈도우에 맞게 조절   │
```

```
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

2.2 서브그래프 추출 방법들

```python
# knowledge_graph/subgraph.py
from typing import List, Set, Tuple, Optional
from dataclasses import dataclass
from collections import deque
import networkx as nx

@dataclass
class Subgraph:
    """추출된 서브그래프"""
    nodes: List[Node]
    edges: List[Edge]
    center_nodes: List[str]  # 쿼리의 중심 노드들
    hop_distance: int

    def get_triplets(self) -> List[Triplet]:
        """서브그래프의 모든 트리플렛 반환"""
        node_map = {n.id: n for n in self.nodes}
        triplets = []

        for edge in self.edges:
            if edge.source_id in node_map and edge.target_id in node_map:
                triplets.append(Triplet(
                    subject=node_map[edge.source_id],
                    predicate=edge,
                    object=node_map[edge.target_id],
                ))

        return triplets

class SubgraphExtractor:
    """서브그래프 추출기"""

    def __init__(self, nodes: List[Node], edges: List[Edge]):
        # NetworkX 그래프 생성
        self.G = nx.DiGraph()

        self.node_map = {}
        for node in nodes:
            self.G.add_node(node.id, data=node)
            self.node_map[node.id] = node

        self.edge_map = {}
        for edge in edges:
            self.G.add_edge(edge.source_id, edge.target_id, data=edge)
            self.edge_map[(edge.source_id, edge.target_id)] = edge
```

```python
    def extract_by_entities(self,
                            entity_ids: List[str],
                            max_hops: int = 2,
                            max_nodes: int = 50) -> Subgraph:
        """
        엔티티 기반 서브그래프 추출

        Args:
            entity_ids: 중심 엔티티 ID들
            max_hops: 최대 홉 수 (거리)
            max_nodes: 최대 노드 수
        """
        visited_nodes: Set[str] = set()
        visited_edges: Set[Tuple[str, str]] = set()

        # BFS로 이웃 탐색
        queue = deque()
        for eid in entity_ids:
            if eid in self.G:
                queue.append((eid, 0))
                visited_nodes.add(eid)

        while queue and len(visited_nodes) < max_nodes:
            current_id, hop = queue.popleft()

            if hop >= max_hops:
                continue

            # 나가는 엣지
            for neighbor in self.G.successors(current_id):
                visited_edges.add((current_id, neighbor))
                if neighbor not in visited_nodes:
                    visited_nodes.add(neighbor)
                    queue.append((neighbor, hop + 1))

            # 들어오는 엣지
            for neighbor in self.G.predecessors(current_id):
                visited_edges.add((neighbor, current_id))
                if neighbor not in visited_nodes:
                    visited_nodes.add(neighbor)
                    queue.append((neighbor, hop + 1))

        # 결과 구성
        result_nodes = [self.node_map[nid] for nid in visited_nodes if nid in
self.node_map]
        result_edges = [self.edge_map[e] for e in visited_edges if e in
self.edge_map]

        return Subgraph(
            nodes=result_nodes,
            edges=result_edges,
            center_nodes=entity_ids,
            hop_distance=max_hops,
        )
```

```python
    def extract_by_relation_path(self,
                                 start_id: str,
                                 end_id: str,
                                 max_path_length: int = 4) -> Subgraph:
        """
        두 엔티티 사이의 경로 기반 서브그래프 추출
        """
        try:
            # 최단 경로 찾기
            paths = list(nx.all_simple_paths(
                self.G.to_undirected(),
                start_id,
                end_id,
                cutoff=max_path_length
            ))
        except nx.NetworkXNoPath:
            paths = []

        visited_nodes: Set[str] = set()
        visited_edges: Set[Tuple[str, str]] = set()

        for path in paths[:5]:  # 최대 5개 경로
            for i, node_id in enumerate(path):
                visited_nodes.add(node_id)
                if i < len(path) - 1:
                    next_id = path[i + 1]
                    if self.G.has_edge(node_id, next_id):
                        visited_edges.add((node_id, next_id))
                    elif self.G.has_edge(next_id, node_id):
                        visited_edges.add((next_id, node_id))

        result_nodes = [self.node_map[nid] for nid in visited_nodes if nid in
self.node_map]
        result_edges = [self.edge_map[e] for e in visited_edges if e in
self.edge_map]

        return Subgraph(
            nodes=result_nodes,
            edges=result_edges,
            center_nodes=[start_id, end_id],
            hop_distance=max_path_length,
        )

    def extract_by_relation_type(self,
                                 center_id: str,
                                 relation_types: List[RelationType],
                                 max_hops: int = 2) -> Subgraph:
        """
        특정 관계 타입만 포함하는 서브그래프 추출
        """
        visited_nodes: Set[str] = {center_id}
        visited_edges: Set[Tuple[str, str]] = set()
```

```
        queue = deque([(center_id, 0)])

        while queue:
            current_id, hop = queue.popleft()

            if hop >= max_hops:
                continue

            # 나가는 엣지 (특정 관계만)
            for neighbor in self.G.successors(current_id):
                edge = self.edge_map.get((current_id, neighbor))
                if edge and edge.type in relation_types:
                    visited_edges.add((current_id, neighbor))
                    if neighbor not in visited_nodes:
                        visited_nodes.add(neighbor)
                        queue.append((neighbor, hop + 1))

            # 들어오는 엣지
            for neighbor in self.G.predecessors(current_id):
                edge = self.edge_map.get((neighbor, current_id))
                if edge and edge.type in relation_types:
                    visited_edges.add((neighbor, current_id))
                    if neighbor not in visited_nodes:
                        visited_nodes.add(neighbor)
                        queue.append((neighbor, hop + 1))

        result_nodes = [self.node_map[nid] for nid in visited_nodes]
        result_edges = [self.edge_map[e] for e in visited_edges if e in
self.edge_map]

        return Subgraph(
            nodes=result_nodes,
            edges=result_edges,
            center_nodes=[center_id],
            hop_distance=max_hops,
        )

    def extract_by_embedding_similarity(self,
                                        query_embedding: List[float],
                                        top_k: int = 10,
                                        expand_hops: int = 1) -> Subgraph:
        """
        임베딩 유사도 기반 서브그래프 추출
        """
        import numpy as np

        query_vec = np.array(query_embedding)

        # 모든 노드와 유사도 계산
        similarities = []
        for node_id, node in self.node_map.items():
            if node.embedding:
                node_vec = np.array(node.embedding)
                sim = np.dot(query_vec, node_vec) / (
```

```
                    np.linalg.norm(query_vec) * np.linalg.norm(node_vec)
                )
                similarities.append((node_id, sim))

        # 상위 k개 선택
        similarities.sort(key=lambda x: x[1], reverse=True)
        top_node_ids = [nid for nid, _ in similarities[:top_k]]

        # 선택된 노드들 기준으로 서브그래프 추출
        return self.extract_by_entities(top_node_ids, max_hops=expand_hops)
```

2.3 서브그래프 시각화

```
추출된 서브그래프 예시 (Query: "김철수가 어떤 프로젝트를 관리하나요?")

┌──────────────────────────────────────────────────────────────────────┐
│                    추출된 서브그래프 (2-hop)                          │
│                                                                      │
│                       [ABC Corp]                                     │
│                           │                                          │
│                       WORKS_AT                                       │
│                           │                                          │
│                           ▼                                          │
│     ┌──────────────── [김철수] ────────────────┐                     │
│     │                Tech Lead                  │                    │
│     │                    │                      │                    │
│     │                    │                      │                    │
│  MANAGES              WORKS_ON               WORKS_AT                │
│     │                    │                      │                    │
│     ▼                    ▼                      ▼                    │
│ [알파 프로젝트]    [베타 프로젝트]         [ABC Corp]                 │
│  status: 진행중     status: 기획                                     │
│     │                    │                                           │
│ USES_TECH            USES_TECH                                       │
│     │                    │                                           │
│     ▼                    ▼                                           │
│ [Python]           [Kubernetes]                                      │
│                                                                      │
│ 노드: 6개, 엣지: 6개, 중심: 김철수, Hop: 2                            │
└──────────────────────────────────────────────────────────────────────┘
```

Part 3: 서브그래프를 컨텍스트로 변환

3.1 변환 전략 비교

```
┌──────────────────────────────────────────────────────────────────────┐
│                    서브그래프 → 컨텍스트 변환 전략                     │
│                                                                      │
│  1. 트리플렛 나열 (Simple Triplets)                                   │
```

```
│     ──────────────────────────                                          │
│     - 김철수 works at ABC Corp                                          │
│     - 김철수 manages 알파 프로젝트                                        │
│     - 김철수 works on 베타 프로젝트                                       │
│     - 알파 프로젝트 uses tech Python                                      │
│     장점: 간단, 빠름 / 단점: 읽기 어려움                                   │
│                                                                         │
│  2. 자연어 문단 (Natural Language)                                        │
│     ──────────────────────────                                          │
│     김철수는 ABC Corp에서 Tech Lead로 근무하고 있습니다.                   │
│     현재 알파 프로젝트를 관리하며, 베타 프로젝트에도 참여하고 있습니다.      │
│     알파 프로젝트는 Python을 사용하고, 베타 프로젝트는 Kubernetes를 사용합니다.│
│     장점: 자연스러움 / 단점: LLM 호출 필요                                 │
│                                                                         │
│  3. 구조화된 형식 (Structured)                                            │
│     ──────────────────────────                                          │
│     ## 김철수 (Person)                                                    │
│     - 역할: Tech Lead                                                    │
│     - 소속: ABC Corp                                                     │
│     - 관리 프로젝트: 알파 프로젝트 (진행중, Python)                        │
│     - 참여 프로젝트: 베타 프로젝트 (기획, Kubernetes)                      │
│     장점: 구조적, 명확 / 단점: 유연성 부족                                 │
│                                                                         │
│  4. 그래프 마크업 (Graph Markup)                                          │
│     ──────────────────────────                                          │
│     <entity type="Person" id="person_1">김철수</entity>는                 │
│     <entity type="Organization">ABC Corp</entity>의                      │
│     <relation>Tech Lead</relation>입니다.                                 │
│     장점: 엔티티 명확 / 단점: 토큰 소모 많음                               │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

3.2 컨텍스트 변환 구현

```python
# knowledge_graph/context_builder.py
from typing import List, Dict, Any, Optional
from dataclasses import dataclass
from enum import Enum
import json

class ContextFormat(Enum):
    TRIPLETS = "triplets"           # 단순 트리플렛
    NATURAL = "natural"             # 자연어 문단
    STRUCTURED = "structured"       # 마크다운 구조
    GRAPH_MARKUP = "graph_markup"   # XML 스타일 마크업
    JSON = "json"                   # JSON 형식

@dataclass
class GraphContext:
    """변환된 그래프 컨텍스트"""
    text: str
```

```python
    format: ContextFormat
    token_estimate: int
    source_subgraph: Subgraph
    metadata: Dict[str, Any]

class ContextBuilder:
    """서브그래프를 LLM 컨텍스트로 변환"""

    def __init__(self,
                 max_tokens: int = 2000,
                 language: str = 'ko'):
        self.max_tokens = max_tokens
        self.language = language

        # 관계 타입 → 자연어 매핑
        self.relation_templates = {
            RelationType.WORKS_AT: "{subj}은(는) {obj}에서 근무합니다",
            RelationType.MANAGES: "{subj}은(는) {obj}을(를) 관리합니다",
            RelationType.WORKS_ON: "{subj}은(는) {obj}에 참여하고 있습니다",
            RelationType.REPORTS_TO: "{subj}은(는) {obj}에게 보고합니다",
            RelationType.USES_TECH: "{subj}은(는) {obj}을(를) 사용합니다",
            RelationType.REQUIRES: "{subj}은(는) {obj}을(를) 필요로 합니다",
            RelationType.AUTHORED: "{subj}은(는) {obj}을(를) 작성했습니다",
            RelationType.DESCRIBES: "{subj}은(는) {obj}에 대해 설명합니다",
            RelationType.REFERENCES: "{subj}은(는) {obj}을(를) 참조합니다",
            RelationType.LOCATED_IN: "{subj}은(는) {obj}에 위치합니다",
            RelationType.PART_OF: "{subj}은(는) {obj}의 일부입니다",
            RelationType.RELATED_TO: "{subj}은(는) {obj}와(과) 관련됩니다",
        }

    def build_context(self,
                      subgraph: Subgraph,
                      format: ContextFormat = ContextFormat.STRUCTURED,
                      prioritize_center: bool = True) -> GraphContext:
        """
        서브그래프를 컨텍스트로 변환

        Args:
            subgraph: 변환할 서브그래프
            format: 출력 형식
            prioritize_center: 중심 노드 우선 정렬
        """
        if format == ContextFormat.TRIPLETS:
            text = self._build_triplets(subgraph)
        elif format == ContextFormat.NATURAL:
            text = self._build_natural(subgraph, prioritize_center)
        elif format == ContextFormat.STRUCTURED:
            text = self._build_structured(subgraph, prioritize_center)
        elif format == ContextFormat.GRAPH_MARKUP:
            text = self._build_markup(subgraph)
        elif format == ContextFormat.JSON:
            text = self._build_json(subgraph)
        else:
            text = self._build_structured(subgraph, prioritize_center)
```

```
        # 토큰 추정 (대략 4글자 = 1토큰)
        token_estimate = len(text) // 4

        # 길이 초과 시 잘라내기
        if token_estimate > self.max_tokens:
            text = self._truncate_context(text, subgraph, format)
            token_estimate = len(text) // 4

        return GraphContext(
            text=text,
            format=format,
            token_estimate=token_estimate,
            source_subgraph=subgraph,
            metadata={
                'node_count': len(subgraph.nodes),
                'edge_count': len(subgraph.edges),
                'center_nodes': subgraph.center_nodes,
            }
        )

    def _build_triplets(self, subgraph: Subgraph) -> str:
        """트리플렛 형식"""
        triplets = subgraph.get_triplets()
        lines = []

        for t in triplets:
            relation = t.predicate.type.value.replace('_', ' ').lower()
            lines.append(f"- {t.subject.name} {relation} {t.object.name}")

        return "관계 정보:\n" + '\n'.join(lines)

    def _build_natural(self, subgraph: Subgraph, prioritize_center: bool) -> str:
        """자연어 문단 형식"""
        node_map = {n.id: n for n in subgraph.nodes}

        # 중심 노드부터 처리
        if prioritize_center:
            ordered_nodes = []
            center_set = set(subgraph.center_nodes)
            ordered_nodes.extend([n for n in subgraph.nodes if n.id in
center_set])
            ordered_nodes.extend([n for n in subgraph.nodes if n.id not in
center_set])
        else:
            ordered_nodes = subgraph.nodes

        paragraphs = []
        processed_edges = set()

        for node in ordered_nodes:
            sentences = []

            # 노드 소개
```

```
            props_desc = self._describe_properties(node)
            if props_desc:
                sentences.append(f"{node.name}은(는) {node.type.value}입니다.
{props_desc}")

            # 관련 관계 설명
            for edge in subgraph.edges:
                edge_key = (edge.source_id, edge.target_id)
                if edge_key in processed_edges:
                    continue

                if edge.source_id == node.id:
                    template = self.relation_templates.get(edge.type)
                    if template:
                        target = node_map.get(edge.target_id)
                        if target:
                            sentences.append(template.format(
                                subj=node.name,
                                obj=target.name
                            ))
                            processed_edges.add(edge_key)

            if sentences:
                paragraphs.append(' '.join(sentences))

        return '\n\n'.join(paragraphs)

    def _build_structured(self, subgraph: Subgraph, prioritize_center: bool) ->
str:
        """마크다운 구조 형식"""
        node_map = {n.id: n for n in subgraph.nodes}

        # 노드 타입별 그룹핑
        nodes_by_type: Dict[NodeType, List[Node]] = {}
        for node in subgraph.nodes:
            if node.type not in nodes_by_type:
                nodes_by_type[node.type] = []
            nodes_by_type[node.type].append(node)

        sections = []

        # 중심 노드 먼저
        if prioritize_center:
            center_nodes = [node_map[cid] for cid in subgraph.center_nodes if cid
in node_map]
            if center_nodes:
                sections.append("## 주요 엔티티\n")
                for node in center_nodes:
                    sections.append(self._format_node_section(node, subgraph,
node_map))

        # 타입별 노드
        for node_type, nodes in nodes_by_type.items():
            non_center = [n for n in nodes if n.id not in subgraph.center_nodes]
```

```
            if non_center:
                sections.append(f"\n## {node_type.value} 정보\n")
                for node in non_center:
                    sections.append(self._format_node_section(node, subgraph,
node_map, brief=True))

        return '\n'.join(sections)

    def _format_node_section(self, node: Node, subgraph: Subgraph,
                             node_map: Dict[str, Node], brief: bool = False) ->
str:
        """단일 노드 섹션 포맷팅"""
        lines = [f"### {node.name}"]

        # 속성
        if node.properties and not brief:
            for key, value in node.properties.items():
                lines.append(f"- {key}: {value}")

        # 관계
        outgoing = []
        incoming = []

        for edge in subgraph.edges:
            if edge.source_id == node.id:
                target = node_map.get(edge.target_id)
                if target:
                    rel = edge.type.value.replace('_', ' ')
                    outgoing.append(f"{rel} → {target.name}")
            elif edge.target_id == node.id:
                source = node_map.get(edge.source_id)
                if source:
                    rel = edge.type.value.replace('_', ' ')
                    incoming.append(f"{source.name} → {rel}")

        if outgoing:
            lines.append("- 관계: " + ", ".join(outgoing[:5]))
        if incoming and not brief:
            lines.append("- 참조됨: " + ", ".join(incoming[:3]))

        return '\n'.join(lines) + '\n'

    def _build_markup(self, subgraph: Subgraph) -> str:
        """XML 스타일 마크업 형식"""
        node_map = {n.id: n for n in subgraph.nodes}
        lines = ["<knowledge_graph>"]

        # 노드
        lines.append("  <entities>")
        for node in subgraph.nodes:
            props = ' '.join(f'{k}="{v}"' for k, v in node.properties.items())
            lines.append(f'    <entity type="{node.type.value}" id="{node.id}"
{props}>{node.name}</entity>')
        lines.append("  </entities>")
```

```
        # 관계
        lines.append("  <relations>")
        for edge in subgraph.edges:
            src = node_map.get(edge.source_id)
            tgt = node_map.get(edge.target_id)
            if src and tgt:
                lines.append(f'    <relation type="{edge.type.value}">')
                lines.append(f'      <subject>{src.name}</subject>')
                lines.append(f'      <object>{tgt.name}</object>')
                lines.append('    </relation>')
        lines.append("  </relations>")

        lines.append("</knowledge_graph>")
        return '\n'.join(lines)

    def _build_json(self, subgraph: Subgraph) -> str:
        """JSON 형식"""
        data = {
            'nodes': [n.to_dict() for n in subgraph.nodes],
            'edges': [e.to_dict() for e in subgraph.edges],
            'center_nodes': subgraph.center_nodes,
        }
        return json.dumps(data, ensure_ascii=False, indent=2)

    def _describe_properties(self, node: Node) -> str:
        """노드 속성 설명"""
        if not node.properties:
            return ""

        descriptions = []
        for key, value in node.properties.items():
            if key == 'role':
                descriptions.append(f"역할은 {value}")
            elif key == 'status':
                descriptions.append(f"상태는 {value}")
            elif key == 'department':
                descriptions.append(f"부서는 {value}")
            elif key == 'experience':
                descriptions.append(f"경력 {value}년")
            else:
                descriptions.append(f"{key}: {value}")

        return ', '.join(descriptions) + '입니다.' if descriptions else ''

    def _truncate_context(self, text: str, subgraph: Subgraph,
                          format: ContextFormat) -> str:
        """컨텍스트 길이 조절"""
        target_chars = self.max_tokens * 4

        if len(text) <= target_chars:
            return text

        # 중심 노드 관련 내용 우선 보존
```

```
        # 간단히 앞부분 자르기 (실제로는 더 정교한 로직 필요)
        return text[:target_chars] + "\n... (이하 생략)"
```

3.3 컨텍스트 변환 예시

```
# 사용 예시
extractor = SubgraphExtractor(nodes, edges)
builder = ContextBuilder(max_tokens=1000, language='ko')

# 서브그래프 추출
subgraph = extractor.extract_by_entities(['person_1'], max_hops=2)

# 다양한 형식으로 변환
print("=" * 60)
print("1. 트리플렛 형식")
print("=" * 60)
ctx = builder.build_context(subgraph, ContextFormat.TRIPLETS)
print(ctx.text)

print("\n" + "=" * 60)
print("2. 자연어 형식")
print("=" * 60)
ctx = builder.build_context(subgraph, ContextFormat.NATURAL)
print(ctx.text)

print("\n" + "=" * 60)
print("3. 구조화된 형식")
print("=" * 60)
ctx = builder.build_context(subgraph, ContextFormat.STRUCTURED)
print(ctx.text)
```

출력 예시:

```
============================================================
1. 트리플렛 형식
============================================================
관계 정보:
- 김철수 works at ABC Corp
- 김철수 manages 알파 프로젝트
- 김철수 works on 베타 프로젝트
- 알파 프로젝트 uses tech Python
- 베타 프로젝트 uses tech Kubernetes
- Python requires FastAPI

============================================================
2. 자연어 형식
============================================================
김철수은(는) Person입니다. 역할은 Tech Lead, 부서는 개발팀, 경력 8년입니다.
김철수은(는) ABC Corp에서 근무합니다. 김철수은(는) 알파 프로젝트을(를) 관리합니다.
```

```
김철수은(는) 베타 프로젝트에 참여하고 있습니다.

알파 프로젝트은(는) Project입니다. 상태는 진행중입니다.
알파 프로젝트은(는) Python을(를) 사용합니다.

베타 프로젝트은(는) Project입니다. 상태는 기획입니다.
베타 프로젝트은(는) Kubernetes을(를) 사용합니다.

============================================================
3. 구조화된 형식
============================================================
## 주요 엔티티

### 김철수
- role: Tech Lead
- department: 개발팀
- experience: 8
- 관계: WORKS AT → ABC Corp, MANAGES → 알파 프로젝트, WORKS ON → 베타 프로젝트

## Project 정보

### 알파 프로젝트
- 관계: USES TECH → Python

### 베타 프로젝트
- 관계: USES TECH → Kubernetes

## Technology 정보

### Python
- 관계: REQUIRES → FastAPI

### Kubernetes
- 관계: REQUIRES → Docker
```

Part 4: 전체 파이프라인

4.1 쿼리 → 엔티티 → 서브그래프 → 컨텍스트 → LLM

```python
# knowledge_graph/rag_pipeline.py
from typing import List, Optional, Tuple
import re

class KnowledgeGraphRAG:
    """Knowledge Graph 기반 RAG 파이프라인"""

    def __init__(self,
                 nodes: List[Node],
                 edges: List[Edge],
                 llm_client,
                 embedder=None):
```

```python
        self.extractor = SubgraphExtractor(nodes, edges)
        self.builder = ContextBuilder(max_tokens=2000)
        self.llm_client = llm_client
        self.embedder = embedder

        # 엔티티 이름 → ID 매핑
        self.name_to_id = {n.name.lower(): n.id for n in nodes}
        self.nodes = {n.id: n for n in nodes}

    def query(self,
              question: str,
              context_format: ContextFormat = ContextFormat.STRUCTURED,
              max_hops: int = 2) -> Tuple[str, GraphContext]:
        """
        질문에 대해 KG 기반 RAG 수행

        Returns:
            (answer, context): 답변과 사용된 컨텍스트
        """
        # 1. 엔티티 추출
        entities = self._extract_entities(question)

        if not entities:
            # 엔티티 없으면 임베딩 기반 검색
            if self.embedder:
                query_emb = self.embedder([question])[0]
                subgraph = self.extractor.extract_by_embedding_similarity(
                    query_emb, top_k=5, expand_hops=1
                )
            else:
                return "관련 정보를 찾을 수 없습니다.", None
        else:
            # 2. 서브그래프 추출
            subgraph = self.extractor.extract_by_entities(
                entities, max_hops=max_hops
            )

        # 3. 컨텍스트 생성
        context = self.builder.build_context(subgraph, context_format)

        # 4. LLM에 질문
        answer = self._ask_llm(question, context.text)

        return answer, context

    def _extract_entities(self, question: str) -> List[str]:
        """질문에서 엔티티 추출 (간단한 방식)"""
        found_entities = []
        question_lower = question.lower()

        # 정확한 이름 매칭
        for name, entity_id in self.name_to_id.items():
            if name in question_lower:
                found_entities.append(entity_id)
```

```
        # 추가: LLM으로 엔티티 추출 가능
        # extracted = self._llm_extract_entities(question)

        return found_entities

    def _ask_llm(self, question: str, context: str) -> str:
        """LLM에 컨텍스트와 함께 질문"""
        prompt = f"""다음 정보를 바탕으로 질문에 답변해주세요.

[관련 정보]
{context}

[질문]
{question}

[답변]"""

        response = self.llm_client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1000,
            messages=[{"role": "user", "content": prompt}]
        )

        return response.content[0].text


# 사용 예시
import anthropic

client = anthropic.Anthropic()
rag = KnowledgeGraphRAG(nodes, edges, client)

question = "김철수가 관리하는 프로젝트는 무엇이고, 어떤 기술을 사용하나요?"
answer, context = rag.query(question)

print("질문:", question)
print("\n사용된 컨텍스트:")
print(context.text)
print("\n답변:", answer)
```

4.2 결과 예시

```
질문: 김철수가 관리하는 프로젝트는 무엇이고, 어떤 기술을 사용하나요?

사용된 컨텍스트:
## 주요 엔티티

### 김철수
- role: Tech Lead
- department: 개발팀
```

```
- experience: 8
- 관계: WORKS AT → ABC Corp, MANAGES → 알파 프로젝트, WORKS ON → 베타 프로젝트

## Project 정보

### 알파 프로젝트
- status: 진행중
- priority: high
- 관계: USES TECH → Python

### 베타 프로젝트
- status: 기획
- 관계: USES TECH → Kubernetes

## Technology 정보

### Python
- version: 3.11
- 관계: REQUIRES → FastAPI

답변:
김철수가 관리하는 프로젝트는 **알파 프로젝트**입니다.

알파 프로젝트는 현재 '진행중' 상태이며 높은 우선순위를 가지고 있습니다.
이 프로젝트는 **Python 3.11**을 사용하며, Python은 FastAPI를 필요로 합니다.

참고로 김철수는 베타 프로젝트에도 참여하고 있으며, 베타 프로젝트는 Kubernetes를 사용합니다.
```

정리

단계	설명	주요 기법
엔티티 추출	쿼리에서 KG 노드 식별	문자열 매칭, NER, LLM
서브그래프 추출	관련 노드/엣지 수집	BFS, 경로 탐색, 임베딩 유사도
컨텍스트 변환	그래프 → 텍스트	트리플렛, 자연어, 구조화, JSON
LLM 질의	컨텍스트 기반 답변 생성	프롬프트 엔지니어링

컨텍스트 포맷 선택 기준:

- **트리플렛**: 간단한 관계 확인, 디버깅
- **자연어**: 대화형 응답, 사용자 친화적
- **구조화**: 복잡한 정보, 명확한 구분
- **JSON**: API 응답, 프로그래밍 처리